



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

Master Degree in Automation Engineering

Mobile Robotics

Prof. Ing. Luca De Cicco

Simulating Competitive Setpoints Navigation for Twin TurtleBots in Environment with Obstacles

Students:

Alessandra Bonavita

Mariangela Ciuffreda

Giuseppe Quitadamo

Accademic year 2025-2026

Contents

1	Introduction	2
2	Development Environment	2
3	System Architecture	2
4	Package Structure	3
5	Launch File Cascade	5
6	Environment	6
6.1	Physical Model and Odometry Pipeline	6
6.2	Real-Time Visualization in RViz	6
6.2.1	Environment and Robot Rendering	6
6.2.2	Custom Visualization Markers	6
6.2.3	Visual Identification and Robot Labeling in RViz	7
7	TF Frame Management	7
8	Game Logic	8
8.1	Game Master	8
8.1.1	Goal Generation and the Margin Problem	8
8.2	Goal Function and Scoring Strategy	9
9	Simulations	13
9.1	TurtleBot3 Burger	13
9.2	Maps	13
9.3	Nav2 Parameters	13
9.4	Tuning and Results	15
10	Conclusions	19

1 Introduction

This report presents the design and implementation of a **multi-robot competitive simulation system** built on ROS 2 Jazzy and the Nav2 navigation stack. The system simulates two TurtleBot3 Burger robots autonomously competing in a shared arena to collect randomly generated goal points, combining robotics, autonomous navigation, and competitive game logic in a single integrated framework. The primary challenge of this project was enabling two robots to operate simultaneously in the same environment without interference, each with its own isolated communication namespace, localization system, and navigation stack. This required solving several technical problems related to multi-robot namespace management, TF tree isolation, clock synchronization between Gazebo and ROS 2, and Nav2 parameter loading under namespaced conditions. The simulation environment is built on Gazebo Harmonic, with different custom arenas featuring walls, obstacles, and dividing structures that create distinct zones. Each robot is equipped with a 2D LiDAR sensor for obstacle detection and localization. The navigation stack runs a full Nav2 pipeline per robot, including AMCL for localization, NavFn for global path planning, and the MPPI controller for local trajectory execution. On top of the navigation infrastructure, a Game Master node manages the competition logic: generating goals on free map cells, monitoring robot positions, assigning points when a robot reaches a goal, and declaring the winner when all goals are collected. Each robot runs an autonomous Goal Function node that selects the best goal to pursue using a scoring strategy that balances proximity to the goal and competitive advantage over the opponent. The entire system is launched through a cascade of Python launch files, each responsible for a specific subsystem, with carefully timed startup delays to ensure correct initialization order.

2 Development Environment

The project was developed using different host systems depending on the team members' hardware. Some members worked on MacBook Pro computers with Apple Silicon processors (M4 Pro and M5), while others used Windows-based PCs. Since ROS 2 Jazzy requires Ubuntu 24.04, the development environment was standardized through virtualization and containerization. For the Apple Silicon machines, an Ubuntu 24.04 ARM64 virtual machine was created using UTM. Inside the Ubuntu virtual machine, a Docker container based on the official ROS image, ROS 2 Jazzy, was used to encapsulate the entire simulation environment, including Gazebo Harmonic, Nav2, TurtleBot3, and all required dependencies. This approach ensured environment reproducibility, portability across different hardware platforms, and isolation of software dependencies. The Dockerfile installs all the necessary ROS 2 packages and configures the environment through a custom entrypoint script, which sets the required environment variables such as `TURTLEBOT3_MODEL`. Further details on how to run the simulation can be found in the [turtlebot3_multirobot_competition](#) GitHub repository.

3 System Architecture

The system is composed of two main ROS 2 packages working together:

- **ugv_competition**, which contains all the custom nodes, launch files, configuration files and maps developed for this project;
- **turtlebot3_simulations**, which provides the Gazebo simulation environment and has been modified to support multi-robot spawning with isolated namespaces

The overall architecture follows a layered design:

- **Simulation Layer:** Gazebo Harmonic simulates the physical environment, robot dynamics, and sensor data. Two TurtleBot3 Burger robots are spawned with isolated namespaces so that all Gazebo topics are published separately (e.g. `/robot1/scan`, `/robot2/scan`).

- **Bridge Layer:** Two separate `ros_gz_bridge` instances translate messages between the Gazebo middleware and ROS 2 DDS, one per robot, each using a dynamically generated namespaced bridge YAML. A `tf_relay` node collects the namespaced TF topics of each robot and republishes them on the global `/tf` topic required by Nav2.
- **Navigation Layer:** A full Nav2 stack runs independently for each robot.
- **Game Layer:** A Game Master node manages the competition logic, generating goals and tracking scores. Each robot runs a Goal Function node that autonomously selects and pursues the most convenient goal using a scoring strategy.

4 Package Structure

The project is organized as reported in the following page, namely:

- **`docker_ws/`:** Docker infrastructure for building and running the simulation environment. `Dockerfile.robotics2` defines the container image with all ROS 2 and Gazebo dependencies. `build.sh` builds the Docker image. `entrypoint.sh` is the container entrypoint that sources the ROS 2 environment and starts the simulation.
- **`ros_ws/src/ugv_competition/config/`:** Nav2 parameter files, one per robot (`nav2_robot1.yaml`, `nav2_robot2.yaml`), containing all navigation parameters.
- **`ros_ws/src/ugv_competition/launch/`:** Launch files and utilities. `metric.launch.py` is the main entry point.
- **`ros_ws/src/ugv_competition/maps/`:** Occupancy grid maps of the competition arena in PGM format with its YAML descriptor. Two different maps are used for the simulations called `custom` and `symmetric`.
- **`ros_ws/src/ugv_competition/rviz/`:** RViz configuration file for visualizing the map, robot poses, and goal markers during the competition.
- **`ros_ws/src/ugv_competition/ugv_competition/`:** Python nodes. `game_master.py` manages the competition logic. `goal_function.py` implements the autonomous goal selection strategy. `tf_relay.py` republishes namespaced TF topics to the global `/tf`. `robot_label.py` gives a text label to the robots.
- **`ros_ws/src/ugv_competition/ugv_competition/metrics/`:** file `metrics.py` with the implemented metrics for computing goal score, such as `euclidean` and `manhattan distance`.
- **`ros_ws/src/turtlebot3_simulations/turtlebot3_gazebo/launch/`:** `custom_world.launch.py` launches Gazebo with the competition world. `spawn_two.launch.py` spawns a single robot with its bridge and robot state publisher.
- **`ros_ws/src/turtlebot3_simulations/turtlebot3_gazebo/models/`:** Gazebo SDF models for the arena obstacles (boxes, cylinders) and the TurtleBot3 Burger robot.
- **`ros_ws/src/turtlebot3_simulations/turtlebot3_gazebo/params/`:** Base bridge YAML (`turtlebot3_burger_bridge.yaml`) used as template by `robot_utils.py` to generate namespaced bridge configurations at runtime.
- **`ros_ws/src/turtlebot3_simulations/turtlebot3_gazebo/worlds/`:** `custom_world.world` defines the Gazebo simulation world with arena walls, obstacles, and physics settings.

```
multirobot_competitive_simulation/
├── docker_ws/
│   ├── Dockerfile.robotics2
│   ├── Dockerfile.project
│   ├── build.sh
│   └── entrypoint.sh
├── ros_ws/
│   └── src/
│       ├── ugv_competition/
│       │   ├── config/
│       │   │   ├── nav2_robot1.yaml
│       │   │   └── nav2_robot2.yaml
│       │   ├── launch/
│       │   │   ├── competition.launch.py
│       │   │   ├── navigation_robot.launch.py
│       │   │   └── robot_utils.py
│       │   ├── maps/
│       │   │   ├── custom_map.pgm
│       │   │   ├── custom_map.yaml
│       │   │   ├── symmetric_map.pgm
│       │   │   └── symmetric_map.yaml
│       │   ├── rviz/
│       │   │   └── competition.rviz
│       │   ├── ugv_competition/
│       │   │   ├── game_master.py
│       │   │   ├── goal_function.py
│       │   │   ├── tf_relay.py
│       │   │   └── metrics/
│       │   │       └── metrics.py
│       └── turtlebot3_simulations/
│           ├── turtlebot3_gazebo/
│           │   ├── launch/
│           │   │   ├── custom_world.launch.py
│           │   │   └── spawn_two.launch.py
│           │   ├── models/
│           │   │   ├── big_box_obstacle/
│           │   │   ├── cylinder_obstacle/
│           │   │   ├── setpoint_marker/
│           │   │   ├── small_box_obstacle/
│           │   │   ├── turtlebot3_burger/
│           │   │   └── turtlebot3_burger_cam/
│           │   ├── params/
│           │   │   └── turtlebot3_burger_bridge.yaml
│           │   └── worlds/
│           │       └── custom_world.world
│       ├── runmac.sh
│       ├── run.sh
│       ├── exec.sh
│       └── execmac.sh
```

- **run.sh/ runmac.sh:** Scripts to build and run the Docker container on Windows and macOS (VM) respectively.
- **exec.sh / exemac.sh:** Scripts to open an interactive shell inside the running Docker container on Windows and macOS (VM) respectively.

5 Launch File Cascade

A launch file cascade was structured: the system starts from a single entry point and expands into a tree of launch files. Each file is responsible for a well-defined part of the system and is called at the appropriate time using `TimerActions` to ensure correct startup ordering.

- **metric.launch.py - Main Entry Point:** this is the top-level launch file that orchestrates the entire system. It accepts three configurable arguments:
 - **map_name:** selects the environment (`custom` or `symmetric`);
 - **robot1_metric, robot2_metric:** selects the distance metric for each robot (`euclidean`, `manhattan` and `estimated_time`);
 - **goal_placement:** controls how goals are distributed (`random` or `symmetric`). If not specified, defaults to `symmetric` for the symmetric map and `random` otherwise.
 - **goal_seed:** to ensure reproducibility of experiments, an optional `goal_seed` parameter can be passed to fix the seed used during goal generation. When set to a non-negative integer, the same goal distribution is produced across multiple runs, allowing fair comparison between different metric configurations under identical conditions.

Based on the selected map, the launch file automatically chooses the correct world file, map YAML, and robot spawn positions. It uses `OpaqueFunction` to resolve these choices at runtime. The system is started in the following order:

1. t=0s: Gazebo simulation via the selected world launch file;
2. t=0s: TF Relay node for global /tf republishing;
3. t=7s: RViz with predefined configuration (`competition.rviz`);
4. t=10s: Nav2 for robot1 via `navigation_robot.launch.py`;
5. t=10s: Nav2 for robot2 via `navigation_robot.launch.py`;
6. t=25s: publish initial pose for robot1;
7. t=25s: publish initial pose for robot2;
8. t=40s: Game Master node, with `goal_placement` passed as parameter;
9. t=41s: Robot Label node;
10. t=45s: Goal Function node for robot1, with selected metric;
11. t=45s: Goal Function node for robot2, with selected metric.

The delays are necessary to ensure that Gazebo is fully running before Nav2 starts, and that Nav2 is active before the initial poses are published. The game logic starts only after everything is running.

- **custom_world.launch.py / simmetric_world.launch.py - Gazebo Environment:** these files launch the Gazebo simulation environment. Each starts the Gazebo server (headless, `-s` flag) and client (GUI, `-g` flag) separately, loading the corresponding world file. Both robots are spawned by calling `spawn_two.launch.py` twice with different namespaces and positions.

- `navigation_robot.launch.py` - Nav2 for a Single Robot: this file starts the complete Nav2 stack for one robot. It uses `ParameterFile` to load the robot-specific YAML parameters file (`nav2_robot1.yaml` or `nav2_robot2.yaml`). All nodes are launched inside a `GroupAction` with `PushRosNamespace` to ensure correct topic namespacing. TF topics are explicitly remapped to the global `/tf` and `/tf_static` to ensure all transforms are available system-wide. More details are provided in Section 7.

6 Environment

6.1 Physical Model and Odometry Pipeline

The competitive scenario is simulated using Gazebo Harmonic, integrated with the ROS 2 Jazzy framework. The physical agents utilized in this environment are two TurtleBot3 Burger robots. These differential-drive platforms are modeled with standard kinematic links, including the `base_link`, `base_footprint`, two drive wheels, a caster wheel, and a 2D LiDAR sensor (`base_scan`).

Accurate state estimation is fundamental for autonomous navigation. In this architecture, the odometry is computed directly by the Gazebo physics engine, which simulates the wheel kinematics and ground friction. This simulated odometry is then transmitted to the ROS 2 ecosystem via the `gz-sim-odometry-publisher-system` plugin. The plugin continuously broadcasts the robot's position and velocity on the `/odom` topic linking the `odom` frame to the `base_footprint` frame. This continuous stream of spatial data is what allows the localization and navigation stacks to operate accurately within the simulated world.

6.2 Real-Time Visualization in RViz

To monitor the competition, evaluate the robots' decision-making, and debug the navigation stack in real-time, RViz is configured as the primary graphical user interface. The visualization environment is anchored to a global `map` frame and integrates several data streams.

6.2.1 Environment and Robot Rendering

The static layout of the arena is rendered by subscribing to the `OccupancyGrid` published on the `/robot1/map` topic. The 3D models of the robots are visualized by subscribing to their respective `/robot_description` topics. To prevent coordinate conflicts in the shared environment, the system utilizes TF prefixes (`robot1` and `robot2`), ensuring that the physical meshes are correctly positioned according to their localized poses.

6.2.2 Custom Visualization Markers

Standard RViz plugins are insufficient for tracking the specific state of a multi-agent competition. Therefore, the system leverages the ROS 2 `Marker` and `MarkerArray` protocols to publish custom, dynamic visual elements directly into the 3D space:

- **Winner Banner:** Upon the conclusion of the match a text marker is displayed in the middle of the arena. This banner announces the winning robot or declares a draw, accompanied by the final score, ensuring the outcome of the competition is immediately visible.
- **Navigation Paths:** The planned trajectories generated by the Nav2 global and local planners are visualized in real-time. This allows for the continuous monitoring of the routing decisions, enabling the evaluation of how the robots navigate around obstacles and avoid each other.

By combining the physical fidelity of Gazebo with this highly customized RViz interface, the system provides a comprehensive and transparent view of the multi-robot competitive dynamics.

6.2.3 Visual Identification and Robot Labeling in RViz

When robots operate in close proximity or cross paths, their identical visual representations make it difficult for the operator to track specific agents, monitor their individual states, or debug localization issues.

To overcome this limitation, a dedicated `robot_label` node was developed to provide real-time visual identification. The node subscribes to the continuous pose estimation streams (`/robot1/amcl_pose` and `/robot2/amcl_pose`) provided by the AMCL localization system.

Instead of modifying the underlying 3D models, the system leverages the ROS 2 `visualization_msgs/MarkerArray` protocol. For each robot, the node generates a `TEXT` marker. The spatial coordinates of the marker are dynamically updated to match the robot's current position (x, y), with a fixed vertical offset (z_{offset}) applied to ensure the text floats clearly above the physical chassis:

$$z_{label} = z * robot + z_{offset} \quad (1)$$

To further enhance visual distinction, the markers are assigned unique color profiles (e.g., Blue for Robot 1, Red for Robot 2) using standard RGBA (Red, Green, Blue, Alpha) color space values. By publishing these markers at a high frequency (10 Hz), the labels smoothly track the robots' movements, providing the operator with an intuitive, unambiguous, and real-time visual reference within the RViz environment.

7 TF Frame Management

The project relies on the Nav2 framework, the standard navigation stack for ROS 2. It is responsible for path planning, localization, obstacle avoidance, and motion control, enabling the robot to navigate within the simulated environment autonomously. During the development process, special attention was required to correctly manage the different coordinate frames used by ROS 2 and the TF2 transformation system. In particular, several challenges were related to the transformations between the `map`, `odom`, and `base_link` frames for both robots. This issue arose because Nav2 expects TF transformations to be published on the global `/tf` and `/tf_static` topics. However, the system was designed around a namespace-based architecture to support multiple robots, requiring additional logic to correctly handle and remap TF data while preserving namespace isolation.

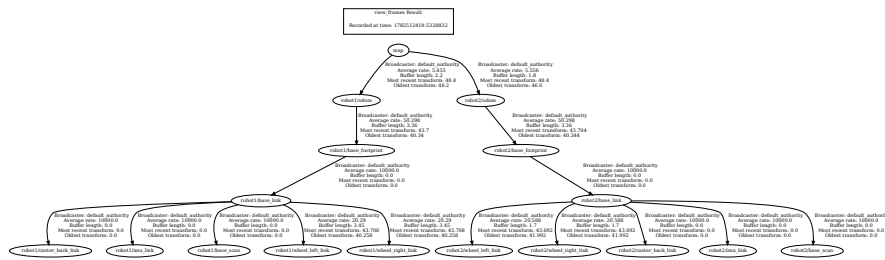


Figure 1: *TF Tree frame*: to view frame hierarchy it was use `ros2 run tf2_tools view_frames` command on terminal

Two additional Python files were added with functions to handle this aspect:

- `robot_utils.py`: this module provides two critical utility functions:
 1. `load_sdf_with_namespace(model_path, namespace)`: patches the TurtleBot3 SDF model file at runtime to inject the robot namespace into all Gazebo topic names. Without this, both robots would publish on the same topics (`/scan`, `/odom`, `/tf`) causing conflicts. The function replaces strings like `<topic>scan</topic>` with `<topic>/robot1/scan</topic>`, and similarly for `cmd_vel`, `odom`, `tf` and `joint_states`.

2. `create_namespaced_bridge_yaml(base_yaml_path, namespace, include_clock=True)`: creates a temporary bridge YAML file in `/tmp/` with all topic names prefixed with the robot namespace. The `include_clock` parameter controls whether the `/clock` topic is included — it must be set to `False` for `robot2` to prevent two `ros_gz_bridge` instances from publishing on `/clock` simultaneously, which caused severe clock instability (hundreds of 'jump back in time' errors per second occurred if not configured).
- `tf_relay.py`: this node subscribes to the namespaced `/tf` topics of each robot (`/robot1/tf`, `/robot2/tf`) and republishes all transforms on the global `/tf` topic. This is necessary because Nav2 nodes remap their `tf` topic to `/tf` using `remappings=[('tf', '/tf')]`, but when running under `PushRosNamespace` the remapping would produce `/robot1/tf`. The `tf_relay` ensures all transforms from both robots are available in a single global TF tree, which is required for AMCL and the costmaps to function correctly. The node also republishes scan messages with namespace-prefixed frame IDs to ensure that the laser scan frame (`robot1/base_scan`) matches the TF tree. This node approach was adapted from the [arshadlab/tb3_multi_robot](#) repository (Apache License 2.0).

8 Game Logic

This section describes the game logic implemented in the simulation, covering the `game_master` node, which acts as the central arbiter of the competition, and the `goal_function` node, which implements the autonomous decision-making strategy for each robot.

8.1 Game Master

The `game_master` node is the central arbiter of the competition. It coordinates goal generation, scoring, and game termination.

Spatial Constraints and Path Evaluation

In a multi-robot competitive environment, the system must guarantee that objectives are physically reachable and that the robots can evaluate their paths efficiently. This section details the strategy behind spatial constraints during goal generation and how it evaluates line-of-sight for strategic decision-making.

8.1.1 Goal Generation and the Margin Problem

When generating random goals in an environment it must not place goals inside physical obstacles, too close to walls, or directly on top of the robots' starting positions. If a goal spawns inside an obstacle, the robot's navigation stack will fail to find a valid path. Similarly, if a goal spawns exactly at the robot's initial coordinates, the robot might register it as reached immediately without actually navigating, giving a huge advantage against the opponent. Therefore, the system must ensure that every goal is placed strictly within the free configuration space (C_{free}) and maintains a safe distance from both obstacles and the robots' initial poses.

To solve this, the `game_master` node implements a unified validation function using a rejection sampling approach. A generated candidate coordinate (x, y) is only accepted if it satisfies three strict mathematical conditions.

First, the continuous real-world coordinates are converted into discrete grid indices (c_x, c_y) using the map's origin and resolution (res):

$$c_x = \left\lfloor \frac{x - x_{origin}}{res} \right\rfloor, \quad c_y = \left\lfloor \frac{y - y_{origin}}{res} \right\rfloor \quad (2)$$

To ensure the goal is inside C_{free} and not too close to a wall, the system performs a circular spatial scan around the target coordinate. It calculates a safety margin in grid cells (M_{cells}) based on a physical safety distance (M_{meters}):

$$M_{cells} = \left\lceil \frac{M_{meters}}{res} \right\rceil \quad (3)$$

The algorithm iterates through all cells within this radius. If the Euclidean distance from the center cell (c_x, c_y) to any neighboring cell ($\Delta x, \Delta y$) is less than or equal to M_{cells} , and that cell contains an obstacle (occupancy value > 0), the goal is rejected:

$$\text{If } \sqrt{\Delta x^2 + \Delta y^2} \leq M_{cells} \text{ and Occupancy}(\Delta x, \Delta y) > 0 \implies \text{Reject Goal} \quad (4)$$

Finally, to prevent goals from spawning on the robots, the system dynamically captures the initial AMCL poses of both robots (P_{spawn1} and P_{spawn2}) at the start of the simulation. The candidate coordinate is only accepted if its distance to both spawn points is strictly greater than a defined spawn margin (D_{spawn}):

$$\sqrt{(x - x_{spawn})^2 + (y - y_{spawn})^2} > D_{spawn} \quad \forall \text{ robots} \quad (5)$$

By combining these checks, the system guarantees that all goals are safely placed in navigable areas, completely independent of the specific map layout.

Game loop:

- **Goal monitoring:** for each active goal, checks if robot1 or robot2 is within `GOAL_RADIUS`; if so, the goal is marked inactive and the robot's score is incremented.
- **Visualization:** active goals are published as green cylinders, collected goals as blue (robot1) or red (robot2) via `/game/goal_markers`. This is useful to see the markers colored during the simulation through Rviz.
- **Score broadcasting:** current scores are published on `/game/score`.
- **Game termination:** when all goals are collected, the winner is declared and a text banner is published in RViz via `/game/banner`.

8.2 Goal Function and Scoring Strategy

Each robot runs an independent `goal_function` node that selects and sends navigation goals to Nav2 every 1 second.

Scoring function:

Each candidate goal g is scored as:

$$\text{score}(g) = -\alpha \cdot d_{own} + \beta \cdot (d_{opp} - d_{own}) + \gamma \cdot S_{cluster} - \text{WALL_PENALTY} \quad (6)$$

with $\alpha = 2.0$, $\beta = 6.0$, $\gamma = 2.0$ and `WALL_PENALTY` = 15.0. See Section 9.4 for more details.

The Bresenham Line-of-Sight Check

The Bresenham line-of-sight check casts a ray on the OccupancyGrid between the robot's current position and the candidate goal. If any cell with value > 25 (occupied or inflated with a threshold to avoid artifacts) is encountered along the ray, a penalty of `WALL_PENALTY` = 15.0m is subtracted to the overall score. This mechanism discourages the robot from targeting goals that require crossing walls or dense obstacle regions in a straight line, implicitly prioritizing goals on the same side of the arena. As a result, each robot tends to collect goals in its own zone before moving to the opponent's side, reducing unnecessary path planning failures near obstacles.

To perform this check efficiently in real-time, the system implements a raycasting mechanism based on **Bresenham's Line Algorithm**. Originally developed for computer graphics to draw lines on a pixel grid, Bresenham's algorithm is highly efficient because it determines the points of a line using only integer addition, subtraction, and multiplication, completely avoiding the computational overhead of floating-point operations.

In this robotic context, the pixels are the discrete cells of the Nav2 **OccupancyGrid**. The algorithm casts a virtual ray from the robot's current grid position (x_0, y_0) to the goal's grid position (x_1, y_1) . It calculates the absolute differences in both axes:

$$dx = |x_1 - x_0|, \quad dy = |y_1 - y_0| \quad (7)$$

The algorithm then uses an error accumulation variable (*err*) to decide whether to step horizontally or vertically at each iteration, ensuring the ray perfectly tracks the theoretical line:

$$err = dx - dy \quad (8)$$

During the traversal, the algorithm evaluates the decision parameter (*e2*) at each step to update the coordinates (x_0, y_0) and the error term:

$$e2 = 2 \cdot err \quad (9)$$

$$\text{If } e2 > -dy \implies err = err - dy, \quad x_0 = x_0 + sx \quad (10)$$

$$\text{If } e2 < dx \implies err = err + dx, \quad y_0 = y_0 + sy \quad (11)$$

where *sx* and *sy* are the step directions, either 1 or -1, depending on the start and end positions.

At every discrete cell along this path, the algorithm queries the **OccupancyGrid**. If it encounters a cell with an obstacle, the line-of-sight is immediately marked as blocked, and the wall penalty is applied to the scoring function. If the algorithm successfully reaches the target cell without hitting an obstacle, the path is considered clear. This allows the robots to make highly intelligent, map-aware decisions in milliseconds.

Alternative Evaluation Metrics

To adapt the robot's navigation and decision-making behavior to different environments or constraints, alternative evaluation metrics can be used in place of standard straight-line distance. The following metrics are implemented and defined within **metrics.py**:

- **euclidean_distance**: The standard straight-line distance between the robot's pose and the goal. This is the most common spatial metric, assuming unobstructed and omnidirectional movement.

$$d_{\text{euclidean}}(P, G) = \sqrt{(x_g - x_r)^2 + (y_g - y_r)^2}$$

- **manhattan_distance**: Measures the distance between two points as the sum of the absolute differences of their Cartesian coordinates. This reflects movement constrained to axis-aligned grid paths.

$$d_{\text{manhattan}}(P, G) = |x_g - x_r| + |y_g - y_r|$$

- **estimated_time_to_goal**: A time-based cost metric rather than a purely spatial one. It estimates how long the robot would take to reach the goal by combining rotation time (angular difference divided by maximum angular velocity) and translation time (Euclidean distance divided by maximum linear velocity).

$$ET(P, \theta_r, G) = \frac{\Delta\theta}{\omega_{\max}} + \frac{d_{\text{euclidean}}(P, G)}{v_{\max}}$$

- **cluster_score**: A density-based metric rather than a distance metric. It counts how many other candidate goals lie within a specified radius of the goal being evaluated. This is particularly useful for prioritizing goals in areas with a high density of nearby points of interest (e.g., in exploration or coverage tasks) instead of isolated outliers and should be used exclusively for goal score evaluation.

$$S_{\text{cluster}}(G_i, \mathcal{G}, R) = \sum_{G_j \in \mathcal{G} \setminus \{G_i\}} \mathbb{I}(D_{\text{euclidean}}(G_i, G_j) \leq R)$$

The goal selection logic is entirely contained in the `select_and_send_goal` method. Each iteration first determines the set of reachable goals, scores them, and then applies a priority chain to decide whether and which goal to send to Nav2. More details are provided below.

Goal selection – reachability: In order to find a new goal to reach, the following iterative pipeline is executed:

- Initially, only goals within radius `MAX_GOAL_DISTANCE = 3.0m` are considered
- If no goals are within range, the radius is expanded by 2.0m

Goal selection – navigation state machine: To prevent conflicting commands and ensure the robot understands its current status, the node implements a strict state machine using the `NavState` enum. The permitted states and transitions are:

- **IDLE**: the robot has no active goal and is ready to receive one.
- **SENDING**: an asynchronous request has been sent to Nav2, and the node is waiting for a response.
- **NAVIGATING**: the goal has been accepted, and the robot is currently moving.
- **CANCELLING**: a cancellation request has been sent to Nav2; the node is waiting for confirmation before sending the next goal.
- **RECOVERING**: the robot has detected a deadlock (stuck state) and is executing a backup maneuver.

Asynchronous call management: To manage non-blocking communication with Nav2 and prevent spamming the action server, the script strictly governs asynchronous transitions using the intermediate states (**SENDING** and **CANCELLING**).

- When a goal is chosen, the state shifts to **SENDING** and a future callback (`on_goal_response`) is attached. During this time, the main loop will ignore new goal calculations.
- If a goal needs to be changed, the node enters **CANCELLING** and triggers `cancel_goal_async`. It waits for `on_cancel_done` to confirm the cancellation before it automatically sends the new, pending goal and resumes **NAVIGATING**.
- If an async call hangs and Nav2 fails to respond within 5 seconds while in the **SENDING** state, a timeout is triggered to reset the state back to **IDLE** so the system does not freeze.

Goal selection – priority chain: Once the best candidate goal has been identified by `select_best_goal`, the method applies a priority chain to determine the appropriate action. The chain evaluates a set of mutually exclusive conditions in order of decreasing urgency: each condition is checked only if the previous ones are not satisfied, ensuring that critical situations such as deadlocks or stolen goals are handled before routine goal switching or standard navigation.

1. **Failsafe check**: it first checks if the robot is stuck in the **SENDING** state for more than 5 seconds and resets to **IDLE** if necessary.

2. **Validity check:** the method terminates immediately if the game is over, there are no available goals, or the robot's pose is unknown.
3. **State check:** it blocks execution if the state is anything other than `IDLE` or `NAVIGATING` to ensure Nav2 has time to process previous asynchronous requests.
4. **Deadlock recovery:** while in `NAVIGATING`, if the robot has not moved more than 0.05 m in `stuck_timeout = 12 s`, the current Nav2 goal is cancelled and a backup `cmd_vel` ($v_x = -0.1$ m/s, $\omega_z = 0.5$ rad/s) is published for 2 seconds via a non-blocking timer.
5. **Goal selection:** Only after passing the State check it calculate the `best_goal` based on distances, wall penalties, and opponent advantage.
6. **Execution:**
 - if `IDLE` it checks if the `best_goal` is identical to the last attempted goal within the last 5 seconds (to prevent spamming an unreachable goal) before sending it.
 - if `NAVIGATING` it runs through a hierarchy of cancellation conditions. It will do nothing if the best goal is the same as the current one. Otherwise, it triggers `cancel_and_send` if the current goal was removed from the active goals list, it has been reached physically (with an error less than `GOAL_REACHED_THRESHOLD = 0.30 m`) or a significantly better goal has emerged that surpasses the adaptive threshold.

$$\Delta_{\text{threshold}} = \text{SWITCH_THRESHOLD} + \frac{1}{d_{\text{current}} + 0.1} \quad (12)$$

The term $\frac{1}{d_{\text{current}} + 0.1}$ makes the threshold increase as the robot approaches its current goal – when the robot is nearly at the destination, the score advantage required to justify switching is high, avoiding unnecessary abandonment of almost completed navigations. Conversely, when far from the current goal, the threshold is low and the robot remains reactive to better opportunities.

MPPI Controller

The Model Predictive Path Integral (MPPI) controller is a highly capable sampling-based local planner. It works by generating and evaluating a large number of random control trajectories to find the optimal path. However, running two MPPI instances simultaneously for both robots within the same Docker container introduced severe computational bottlenecks. The CPU was unable to process the required number of trajectory samples within the allocated time step. Consequently, the controller missed its desired loop rate (e.g., 20 Hz), resulting in infinite loop rate warnings, costmap timeouts, and frequent navigation failures.

The computational complexity of the MPPI algorithm scales linearly with the number of sampled trajectories, which is defined by the `batchsize` parameter (N). The total processing time ($T * \text{comp}$) per control cycle can be approximated as:

$$T_{\text{comp}} \approx N \cdot t_{\text{sample}} + t_{\text{overhead}} \quad (13)$$

where t_{sample} is the time required to evaluate a single trajectory and t_{overhead} represents fixed computational costs. When T_{comp} exceeds the controller's time budget ($1/f_{\text{control}}$), the system becomes unstable.

To resolve this without altering the fundamental Nav2 architecture or drastically lowering the control frequency, the `batch_size` was reduced to 1000. This effectively halved the primary computational load, allowing the controller to consistently meet its frequency requirement and eliminating the costmap timeouts.

However, this solution required a deliberate engineering trade-off. By reducing N , the resolution of the sampled trajectory space is decreased. This inherently reduces the MPPI's performance, making the generated paths slightly less optimal and potentially less smooth in highly cluttered environments compared to a higher batch size.

9 Simulations

9.1 TurtleBot3 Burger

The TurtleBot3 Burger is a small differential drive mobile robot developed by ROBOTIS turtlebot3.robotis.com. It is equipped with a 360° 2D LiDAR sensor that provides laser scan data used for both localization (AMCL) and obstacle avoidance (costmaps, collision monitor).

9.2 Maps

Arena maps used in this work are acquired via SLAM, performed using the `slam_toolbox` package while manually teleoperating a single robot through the environment to build an accurate occupancy grid representation. Two custom maps are used to evaluate the implemented framework.

1. *Custom map*: an asymmetric environment featuring heterogeneous obstacles, such as cubic and parallelepiped shapes. This scenario is used as a general-purpose map for tuning the parameters described in the following section 9.3.
2. *Symmetric map*: a symmetric environment designed to evaluate and compare different performance metrics. In this setup, the two robots are initialized in symmetric positions, and the goals are also placed symmetrically. Two distinct *goal function* nodes, each based on a different metric, are initialized through the parameter `robotX_metric`, where `X` is the number of the robot, namely 1 or 2. This configuration enables a systematic comparison of the results obtained during simulation under controlled and symmetric conditions.

Figure 2 shows the *custom map*: the environment is defined as a bounded rectangular arena in Gazebo containing a heterogeneous set of static obstacles. The world includes both cylindrical and box-shaped (parallelepiped) obstacles with non-uniform spatial distribution.

Unlike the symmetric scenario, the obstacle configuration is intentionally asymmetric. In the upper region of the environment, multiple obstacles are placed at irregular positions, including a large rectangular obstacle in the upper-left area and cylindrical obstacles distributed on the right side of the map. In the lower region, a smaller set of box-shaped obstacles is introduced, further increasing spatial variability.

Figure 3 shows the *symmetric map*: the environment consists of a rectangular bounded area populated with cylindrical obstacles arranged in a symmetric configuration with respect to the horizontal axis ($y = 0$). The obstacle layout is organized as a uniform grid in the upper half-plane ($y > 0$), composed of three columns and two rows. The same arrangement is mirrored in the lower half-plane ($y < 0$), ensuring geometric symmetry.

Specifically, obstacle columns are positioned at $x = \{-2.0, 0.0, 2.0\}$ m, while rows are placed at $y = \{1.2, 2.8\}$ m and their symmetric counterparts at $y = \{-1.2, -2.8\}$ m. Each obstacle is a cylinder of radius 0.5 m, resulting in uniform spacing between adjacent objects and a structured, collision-rich environment suitable for comparative navigation experiments.

All obstacles used in the simulation are custom-designed and implemented as static SDF models, defined specifically for this work.

9.3 Nav2 Parameters

The Nav2 parameter tree is the core configuration framework of the ROS 2 Navigation Stack. Since the stack is platform-agnostic by design, parameter tuning is essential to reconcile theoretical path-planning with the physical constraints of the actual hardware, governing sensor processing, footprint representation, and kinematic enforcement. The following parameters are calibrated for the robot's differential drive kinematics and simulation environment:

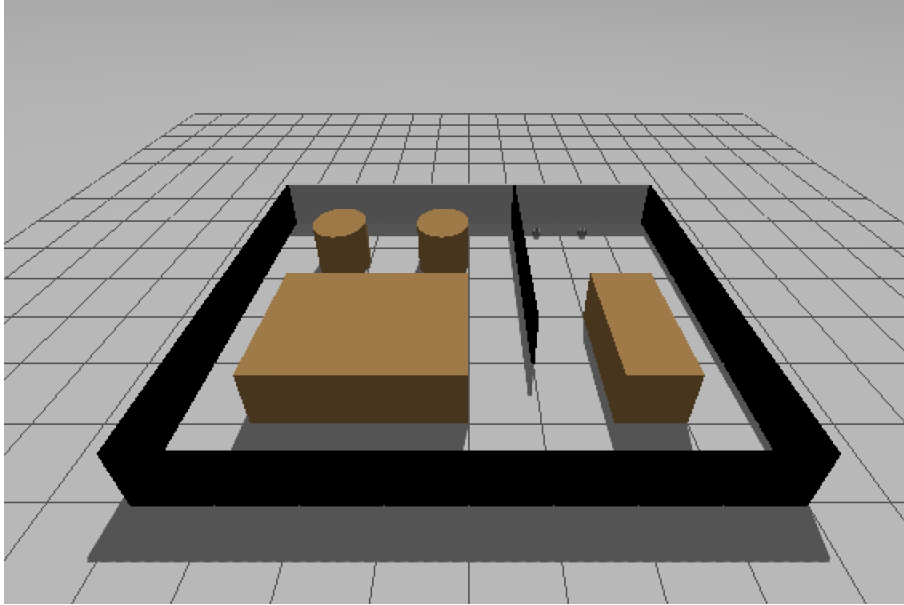


Figure 2: *Custom map (Gazebo simulation):* the environment consists of a bounded rectangular arena populated with heterogeneous obstacles, including both cylindrical and box-shaped objects. The obstacle layout is intentionally asymmetric, with different configurations in the upper and lower regions of the map.

- **Velocity limits.** The velocity constraints are configured to reflect the kinematic model of a differential drive platform. Lateral velocity (`vy_max`) is set to 0.0 m/s, as such models are inherently non-holonomic and incapable of sideways motion. Maximum linear and angular velocities are conservatively bounded (`vx_max` = 0.2 m/s, `vx_min` = -0.2 m/s, `wz_max` = 1.0 rad/s) prioritising controlled, safe manoeuvring over aggressive performance.
- **Costmap inflation.** To improve navigability in confined environments, the `inflation_radius` is substantially reduced on both the local costmap (0.25 m) and the global costmap (0.2 m). This allows the robot to approach obstacles and dynamic entities more closely without triggering planning failures. Additionally, the `cost_scaling_factor` are lowered to ensure a smooth cost gradient around obstacles, avoiding sharp transitions that could destabilise the planner.
- **Collision monitor.** A dedicated `collision_monitor` node is instantiated, configured with a `StopCircle` geometric boundary of 0.5m radius. With its `action_type` set to `stop`, this component acts as a hard safety override: any obstacle violating the defined proximity envelope triggers an immediate halt, independently of the planner state.
- **Goal tolerance and coordination.** The `yaw_goal_tolerance` is set to 2π rad, permitting the robot to reach any waypoint regardless of its final heading. The `xy_goal_tolerance` is tuned to balance positional accuracy against the risk of terminal oscillation near the goal. To maintain coherent behaviour with the higher-level competition logic, the thresholds `GOAL_REACHED_THRESHOLD` (in `goal_function.py`) and `GOAL_RADIUS` (in `game_master.py`) are intentionally set larger (0.3m), ensuring that Nav2 is forced to transition to the next waypoint only after `game_master.py` has explicitly deactivated the current goal, thereby preventing premature switching or oscillation.

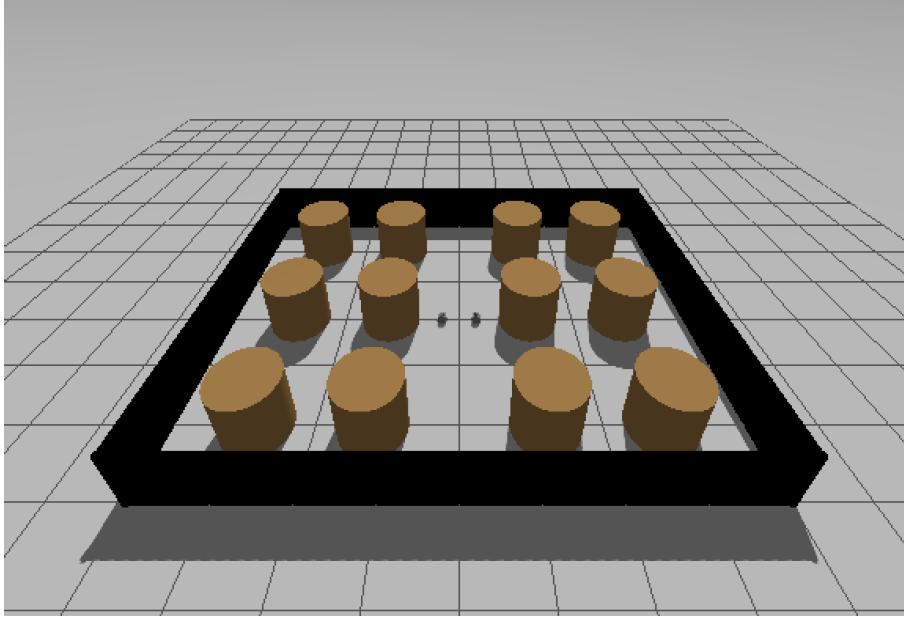


Figure 3: *Symmetric map (Gazebo simulation):* the environment consists of a rectangular bounded area in Gazebo populated with cylindrical obstacles arranged in a symmetric grid with respect to the horizontal axis ($y = 0$). The obstacle layout is mirrored between the upper ($y > 0$) and lower ($y < 0$) regions, ensuring identical spatial structure in both halves of the environment.

9.4 Tuning and Results

All the simulations are performed using a fixed `goal_seed = 100` to guarantee reproducibility. This seed is intentionally selected because, in the symmetric map, it generates goal configurations that place the target positions in particularly challenging locations, such as close to walls and corners. These scenarios require the robots to navigate through narrow spaces while simultaneously coordinating with one another, making them well suited for evaluating both collision avoidance behavior and the impact on overall simulation time under constrained conditions. An example of such a configuration is shown in Figure 4, which displays the global costmap of robot `r1` as visualized in Rviz2 during the simulation.

The tuning procedure was structured in two phases.

In the **first phase**, the parameters `ALPHA`, `BETA`, and `GAMMA` were evaluated on the symmetric map with `NUM_GOALS = 10` and symmetric goal placement, resulting in 6 goals spawned on the map, due to the maximum number of iterations. A baseline configuration was first tested by setting each parameter to 1 while the others were set to 0 (e.g. `ALPHA = 1`, `BETA = 0`, `GAMMA = 0`). Subsequently, all combinations of `ALPHA`, `BETA`, and `GAMMA` with values in $\{2, 4, 6\}$ were evaluated. For each configuration, the total simulation time required to collect all goals was recorded as the primary performance metric. The parameter combination yielding the shortest simulation time was selected as the optimal configuration, since it reflects the most efficient joint behavior of the two robots.

In the **second phase**, the best parameter configuration identified in the first phase was used to run additional simulations with `NUM_GOALS = 15` on the same custom map. This allowed verification that the selected parameters generalize well to a higher number of goals, stressing the system further in terms of coordination and navigation efficiency. Three metric configurations were evaluated: `Manhattan` versus `Euclidean`, `Manhattan` versus `Time-to-Goal`, and `Euclidean` versus `Time-to-Goal`, in order to assess whether the choice of distance metric influences the overall performance under the tuned parameter configuration.

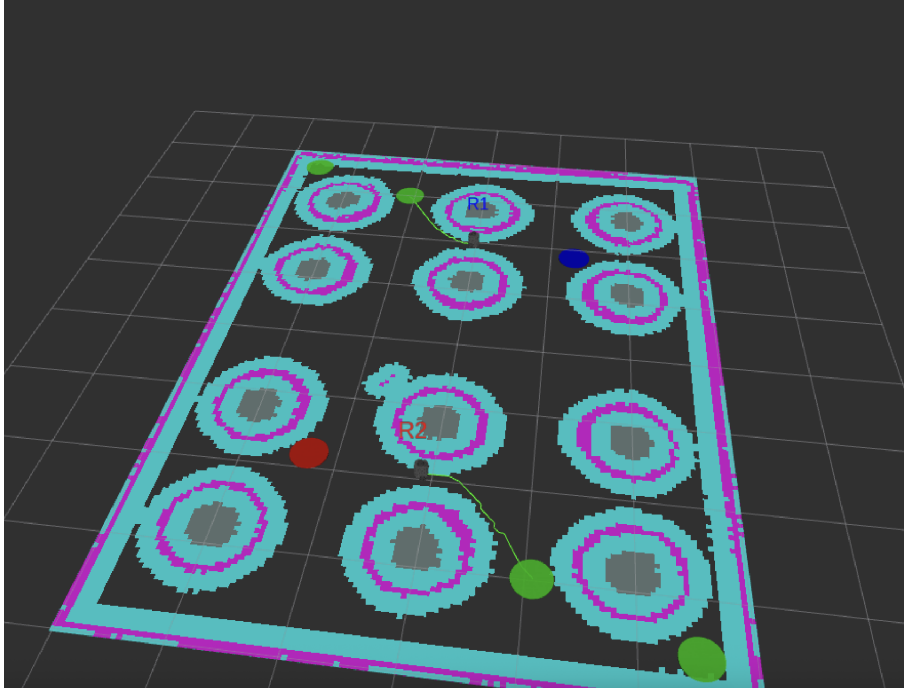


Figure 4: Global costmap of Robot `r1` displayed in RViz during a simulation. The selected `goal_seed = 100` generates a symmetric goal configuration in which several targets are located close to walls and corners, creating constrained navigation scenarios that stress both inter-robot coordination and path planning.

As shown in Table 1, all tested configurations consistently produce a draw (3–3), confirming that the symmetric map and goal placement guarantee fairness between the two robots regardless of the metric used. The main discriminating factor between configurations is therefore the total simulation time, which reflects how efficiently the two robots collectively explore the arena and collect goals.

The baseline configurations with only one active parameter confirm that relying solely on γ (cluster score) leads to the worst performance, with times reaching up to 166 s, as the robots tend to converge toward the same dense goal cluster rather than distributing their effort across the arena. Configurations with only α or β active perform better but still lack the coordination needed to minimize total time.

Among the combined configurations, a high value of β consistently yields shorter simulation times across all metric pairs, as it encourages each robot to prioritize goals where it has a competitive advantage over the opponent, naturally distributing the workload. Increasing α beyond 2 tends to make the robots too greedy toward nearby goals, sometimes causing both to compete for the same target and increasing total time. A high γ introduces instability in some configurations, as the cluster bonus can override the competitive term and attract both robots to the same zone.

The configuration ($\alpha = 2, \beta = 6, \gamma = 2$) was selected as the best tradeoff: it achieves consistently low simulation times across all three metric combinations (86.6 s, 84.6 s, and 89.6 s respectively), without showing the instability observed in configurations with higher α or γ . The dominant β term ensures that each robot pursues goals where it holds a positional advantage over the opponent, while the moderate α and γ terms provide a secondary preference for closer and clustered goals without disrupting the competitive balance.

In the second phase, the optimal parameter configuration ($\alpha = 2, \beta = 6, \gamma = 2$) selected in the first phase was used to run simulations on the custom map with `NUM_GOALS = 15` and `goal_seed`

Table 1: Simulation time (seconds) for different (α, β, γ) configurations with `goal_seed = 100`, symmetric map, 6 symmetric goals. The score is always 3-3 for all configurations. The selected configuration $(\alpha = 2, \beta = 6, \gamma = 2)$ is highlighted.

(α, β, γ)	R1: Manhattan, R2: Time	R1: Euclidean, R2: Time	R1: Euclidean, R2: Manhattan
(1, 0, 0)	93.2	94.4	88.4
(0, 1, 0)	89.8	92.8	122.2
(0, 0, 1)	148.8	97.6	166.4
(2, 2, 2)	87.0	90.0	110.4
(4, 2, 2)	94.6	91.4	138.0
(6, 2, 2)	88.6	88.8	135.8
(2, 4, 2)	96.0	87.4	101.6
(4, 4, 2)	85.6	89.6	95.0
(6, 4, 2)	90.0	84.2	90.6
(2, 6, 2)	86.6	84.6	89.6
(4, 6, 2)	96.0	83.8	86.4
(6, 6, 2)	91.6	87.0	83.0
(2, 2, 4)	98.2	83.4	107.0
(4, 2, 4)	86.6	94.0	138.4
(6, 2, 4)	87.0	90.4	131.4
(2, 4, 4)	99.6	85.0	109.2
(4, 4, 4)	87.2	93.4	140.2
(6, 4, 4)	91.2	99.8	92.4
(2, 6, 4)	86.0	105.4	102.8
(4, 6, 4)	82.8	89.2	89.4
(6, 6, 4)	89.4	92.6	93.2
(2, 2, 6)	90.6	82.4	104.2
(4, 2, 6)	90.8	85.2	129.0
(6, 2, 6)	98.4	96.2	137.8
(2, 4, 6)	90.6	88.4	95.0
(4, 4, 6)	89.6	99.8	101.0
(6, 4, 6)	85.2	89.8	87.4
(2, 6, 6)	92.2	84.6	90.4
(4, 6, 6)	89.4	83.8	91.0
(6, 6, 6)	87.2	86.8	91.6

= 100 (Figure 5). Since the custom map is not symmetric, each metric pair was tested in both directions to account for the different starting positions and obstacle distributions faced by each robot. The following configurations were evaluated:

- R1: Euclidean vs R2: Manhattan and its inverse
- R1: Euclidean vs R2: Time-to-Goal and its inverse
- R1: Manhattan vs R2: Time-to-Goal and its inverse

As shown in Table 2, the results on the custom map highlight the influence of both the metric choice and the starting position on the overall game outcome. Robot1 starts at a position which places it in a structurally disadvantaged zone — closer to the central wall and with fewer immediately reachable goals compared to robot2, which starts in a more open area with better access to the lower part of the arena.

Despite this disadvantage, the **Euclidean** metric allows robot1 to recover effectively. By minimizing raw distance to the nearest goal, it drives robot1 to quickly explore the lower zone of the map, where several goals are clustered, compensating for the less favorable starting position. This is confirmed by the **Euclidean** vs **Manhattan** result, where robot1 wins 9-6 — the only configuration in which robot1 outperforms robot2.

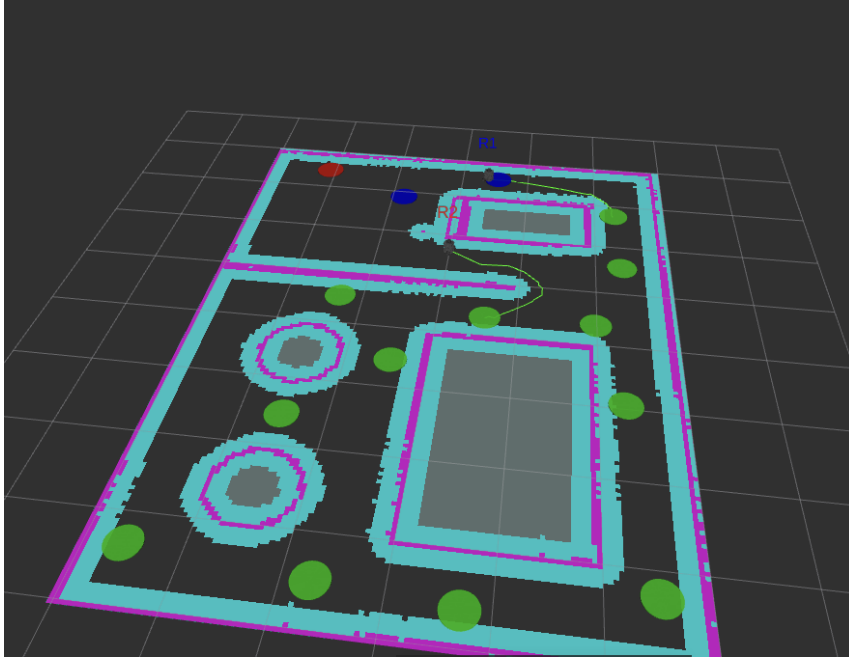


Figure 5: Example of goal placement on the custom map with `goal_seed = 100` and `NUM_GOALS = 15`, visualized in RViz2. Green cylinders represent active goals, while blue and red cylinders indicate goals already collected by robot1 and robot2 respectively. The asymmetric distribution of goals across the arena reflects the non-symmetric structure of the custom map.

The **Time-to-Goal** metric consistently favors robot2 across all configurations where it is assigned as R2, winning 9–6 against both **Euclidean** and **Manhattan**. The most significant result is observed in the **Manhattan** vs **Time-to-Goal** configuration, where robot2 collects 12 out of 15 goals — the largest score difference across all tests. This suggests that **Time-to-Goal** is particularly effective in the custom map, where the obstacle layout makes travel time a more informative cost estimate than raw distance. Conversely, when **Time-to-Goal** is assigned to robot1, its advantage is significantly reduced, as the upper zone presents fewer opportunities where estimated time diverges meaningfully from Euclidean distance.

Overall, the results confirm that in asymmetric environments, the choice of metric interacts strongly with the starting position, and that no single metric dominates unconditionally — the best choice depends on the local structure of the arena assigned to each robot.

Table 2: Simulation results on the custom map with $(\alpha = 2, \beta = 6, \gamma = 2)$, `goal_seed = 100`, `NUM_GOALS = 15`. Each metric pair is tested in both directions to account for the asymmetry of the map.

Configuration	Time (s)	Score (R1 – R2)
R1: Euclidean, R2: Time-to-Goal	159.4	6 – 9
R1: Time-to-Goal, R2: Euclidean	146.6	6 – 9
R1: Manhattan, R2: Time-to-Goal	156.0	6 – 9
R1: Time-to-Goal, R2: Manhattan	193.4	3 – 12
R1: Euclidean, R2: Manhattan	151.6	9 – 6
R1: Manhattan, R2: Euclidean	152.2	6 – 9

10 Conclusions

This work presented the design and implementation of a multi-robot competitive simulation framework built on ROS 2 Jazzy and Gazebo Harmonic, in which two TurtleBot3 Burger robots autonomously compete to collect randomly generated goals in a shared arena.

The main technical contributions include the development of a namespace-based multi-robot architecture with custom TF management, the integration of the Nav2 navigation stack with MPPI controller for each robot, and the design of a competitive game logic with configurable goal selection strategies. The framework supports multiple distance metrics (**Euclidean**, **Manhattan**, **Time-to-Goal**), two arena configurations, symmetric and random goal placement, and reproducible experiments via a fixed random seed.

The experimental evaluation was structured in two phases. In the first phase, the scoring parameters α , β , and γ were tuned on the symmetric map using simulation time as the optimization criterion. The configuration ($\alpha = 2, \beta = 6, \gamma = 2$) was identified as the best tradeoff, consistently achieving low completion times across all metric combinations. The dominant β term encourages each robot to pursue goals where it holds a competitive advantage over the opponent, naturally distributing the workload across the arena.

In the second phase, the selected parameters were validated on the asymmetric custom map with a higher number of goals. The results revealed that the metric choice interacts strongly with the starting position — the **Euclidean** metric allows robot1 to compensate for its structurally disadvantaged starting zone by quickly exploring the lower part of the arena, while **Time-to-Goal** consistently benefits robot2 due to the obstacle layout of its zone.

Future work could explore the use of machine learning-based heuristics to adaptively tune the scoring parameters during the game, as well as the extension of the framework to more than two robots and more complex arena configurations.